MARA Complete Build Guide

Multi-Agent Research Assistant (MARA)

Complete Build Guide — From Zero to Working System

Final-Year B.Tech CSE Project | Primary Project Documentation

---

Table of Contents

1. Project Overview
2. Prerequisites & Skills Checklist
3. System Architecture
4. Technology Stack
5. Environment Setup
6. Phase 1 — Project Scaffolding
7. Phase 2 — Data Ingestion (arXiv / Semantic Scholar)
8. Phase 3 — Retriever Agent (RAG Pipeline)
9. Phase 4 — Planner Agent
10. Phase 5 — Critic Agent (Fact-Checking)
11. Phase 6 — Writer Agent
12. Phase 7 — LangGraph Orchestration (Wiring Everything Together)
13. Phase 8 — Backend API (FastAPI)
14. Phase 9 — Frontend Interface
15. Phase 10 — Deployment (Free Tier)
16. Phase 11 — Evaluation Framework
17. Phase 12 — Testing, Report & Viva Preparation
18. Full Repository Structure
19. Troubleshooting Guide
20. Cost Breakdown
21. Week-by-Week Timeline
22. Base Paper & Further Reading

---

1. Project Overview

What you are building: A system where a user types a research question in plain English and receives a well-organized, **citation-backed** summary of relevant academic literature — generated not by a single AI call, but by **four specialized AI agents** working together:

Agent	Role
Planner	Breaks the user's question into smaller, focused sub-questions
Retriever	Searches indexed academic papers and pulls back relevant text chunks (this is Retrieval-Augmented Generation, or RAG)
Critic	Checks every claim the system is about to make against the retrieved evidence, rejecting anything unsupported
Writer	Composes the final answer, with inline citations pointing to the exact source

Why this matters: A single LLM prompt ("summarize this topic") tends to hallucinate facts and provide no way to verify its claims. By splitting the task into four roles with a verification step in the middle, MARA

produces answers that are traceable back to real sources — the same design principle used in serious research-assistant and enterprise RAG systems.

Primary base paper: *"Expert Multi-Agent Conversational System Using Retrieval-Augmented Generation and Dynamic Text-to-SQL for Government Transparency"* (Flores, Ramirez, Mauricio — IEEE Access, 2025). This paper validates the exact architecture you are building: specialized agents + RAG + an orchestration layer, evaluated using Faithfulness, Answer Relevancy, Answer Similarity, and Answer Correctness — the same metrics you will use in Phase 11.

---

2. Prerequisites & Skills Checklist

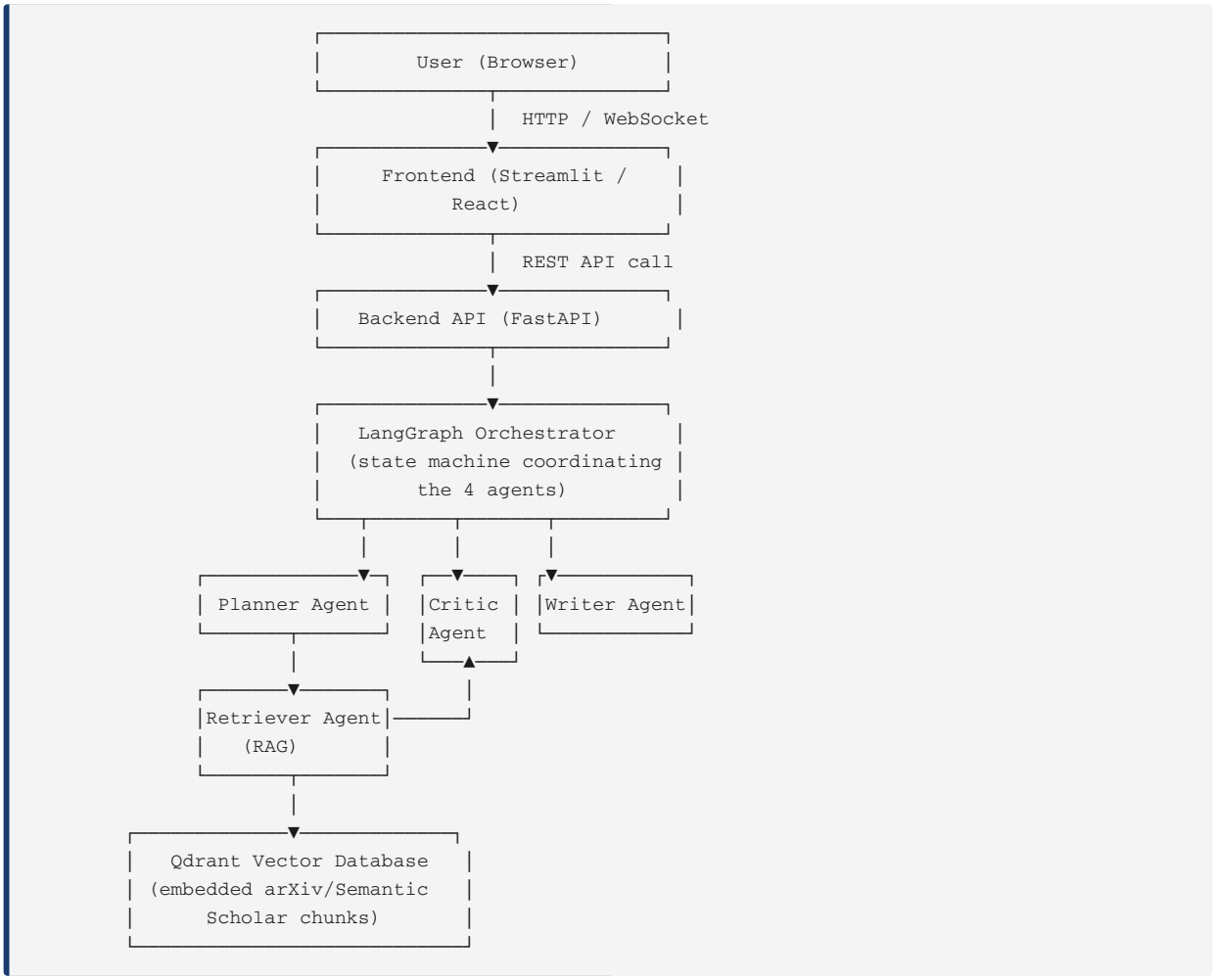
You do **not** need to know all of this before starting — you will learn most of it *while* building (see Section 21). But it helps to know what's coming:

- ☐ Python 3.10+ (comfortable writing functions, classes, using pip)
- ☐ Basic command-line usage (cd, mkdir, running scripts)
- ☐ What an API is and how to make an HTTP request
- ☐ Git basics (clone, add, commit, push) — for version control and submission
- ☐ Willingness to read official documentation (LangGraph, Qdrant, FastAPI all have good docs)

Nice-to-have, not required up front: - Prior exposure to Python virtual environments - Basic HTML/CSS if you want to customize the frontend beyond Streamlit defaults

---

3. System Architecture



Flow of a single query: 1. User submits a research question via the frontend. 2. Backend forwards it to the LangGraph orchestrator. 3. **Planner** decomposes it into 3–5 sub-questions. 4. For each sub-question,

the **Retriever** performs a semantic search against Qdrant and returns the top-k relevant chunks. 5. Draft claims are generated and passed to the **Critic**, which checks each one against retrieved evidence and labels it Supported / Not Supported. 6. Only supported claims move to the **Writer**, which composes the final, cited summary. 7. The response streams back to the frontend with expandable “show sources” sections.

---

4. Technology Stack

Layer	Technology	Why
Orchestration	**LangGraph**	Explicit state-graph model for multi-agent workflows; easier to debug than fully autonomous agent loops
LLM	**Claude API** or **OpenAI GPT-4o-mini** (dev) → stronger model for final demo	Free trial credits available; swap cheap model in during development
Embeddings	**OpenAI** `text-embedding-3-small` or `bge-small-en` (free, local)	Converts text into vectors for semantic search
Vector Database	**Qdrant** (Docker locally, or Qdrant Cloud free tier)	Fast, scalable semantic search, generous free tier
Data Source	**arXiv API** (free, no key) + **Semantic Scholar API** (free, generous limits)	Real academic papers
Backend	**Python + FastAPI**	Async support, automatic OpenAPI docs, easy to deploy free
Frontend	**Streamlit** (fastest) or **React** (more polished)	Streamlit for speed; React if you want a portfolio-grade UI
Hosting	**Render / Railway** (backend), **Vercel / Streamlit Community Cloud** (frontend), **Qdrant Cloud** (vector DB)	All have free tiers sufficient for a student project

---

5. Environment Setup

5.1 Install Python and Create a Virtual Environment

```
python3 --version        # confirm 3.10+
mkdir mara-project && cd mara-project
python3 -m venv venv
source venv/bin/activate         # Windows: venv\Scripts\activate
```

5.2 Install Core Dependencies

```
pip install langgraph langchain langchain-community \
            anthropic openai \
            qdrant-client \
            fastapi uvicorn \
            streamlit \
            requests python-dotenv \
            arxiv semanticscholar \
            pydantic
```

5.3 Get Your API Keys

1. **Anthropic (Claude):** sign up at console.anthropic.com — new accounts typically receive free trial credits.

2. **OpenAI (optional, for embeddings/GPT-4o-mini):** platform.openai.com — also offers free trial credits for new accounts.
3. **Qdrant Cloud (optional, if not running locally):** cloud.qdrant.io — free tier cluster.

Store keys in a `.env` file (never commit this file):

```
ANTHROPIC_API_KEY=your_key_here
OPENAI_API_KEY=your_key_here
QDRANT_URL=your_cluster_url_here
QDRANT_API_KEY=your_key_here
```

Add `.env` to `.gitignore` immediately.

5.4 Run Qdrant Locally (Alternative to Cloud)

```
        docker run -p 6333:6333 qdrant/qdrant
```

This is free and avoids any cloud dependency during development.

---

6. Phase 1 — Project Scaffolding

Create the folder structure before writing any logic:

```
mara-project/
├── .env
├── .gitignore
├── requirements.txt
├── backend/
│   ├── main.py                # FastAPI entry point
│   ├── agents/
│   │   ├── planner.py
│   │   ├── retriever.py
│   │   ├── critic.py
│   │   └── writer.py
│   ├── graph.py                # LangGraph orchestration
│   ├── ingestion/
│   │   └── fetch_papers.py     # arXiv/Semantic Scholar fetch + chunk + embed
│   └── config.py
├── frontend/
│   └── app.py                  # Streamlit UI
├── evaluation/
│   ├── benchmark_queries.json
│   └── evaluate.py
└── README.md
```

Initialize git immediately:

```
        git init
        echo "venv/
        .env
        __pycache__/
        *.pyc" > .gitignore
        git add .
        git commit -m "Initial project scaffolding"
```

---

7. Phase 2 — Data Ingestion (arXiv / Semantic Scholar)

This is your **proof that the data pipeline works** before adding any AI logic on top.

`backend/ingestion/fetch_papers.py`:

```
import arxiv

def fetch_arxiv_papers(query: str, max_results: int = 20):
    """Fetch papers from arXiv matching a query string."""
    search = arxiv.Search(
        query=query,
        max_results=max_results,
        sort_by=arxiv.SortCriterion.Relevance
    )
    papers = []
    for result in search.results():
        papers.append({
            "title": result.title,
            "abstract": result.summary,
            "authors": [a.name for a in result.authors],
            "published": str(result.published),
            "url": result.entry_id,
        })
    return papers

if __name__ == "__main__":
    papers = fetch_arxiv_papers("retrieval augmented generation", max_results=5)
    for p in papers:
        print(p["title"], "-", p["url"])
```

Run it standalone first:

```
python backend/ingestion/fetch_papers.py
```

Checkpoint: you should see 5 real paper titles printed. Do not proceed until this works — everything downstream depends on it.

---

8. Phase 3 — Retriever Agent (RAG Pipeline)

8.1 Chunk and Embed

```
# backend/ingestion/index_papers.py
from qdrant_client import QdrantClient
from qdrant_client.models import Distance, VectorParams, PointStruct
import openai
import os

client = QdrantClient(url=os.getenv("QDRANT_URL"), api_key=os.getenv("QDRANT_API_KEY"))
COLLECTION = "mara_papers"

def chunk_text(text, chunk_size=500):
    words = text.split()
    return [" ".join(words[i:i+chunk_size]) for i in range(0, len(words), chunk_size)]

def embed(text):
    response = openai.embeddings.create(model="text-embedding-3-small", input=text)
    return response.data[0].embedding

def index_papers(papers):
    client.recreate_collection(
        collection_name=COLLECTION,
        vectors_config=VectorParams(size=1536, distance=Distance.COSINE)
    )
    points = []
    idx = 0
    for paper in papers:
        chunks = chunk_text(paper["abstract"])
        for chunk in chunks:
            vector = embed(chunk)
```

```
                    points.append(PointStruct(
                        id=idx,
                        vector=vector,
                        payload={"text": chunk, "title": paper["title"], "url": paper["url"]}
                    ))
                    idx += 1
            client.upsert(collection_name=COLLECTION, points=points)
            print(f"Indexed {len(points)} chunks from {len(papers)} papers.")
```

8.2 The Retriever Function

```
        # backend/agents/retriever.py
        from backend.ingestion.index_papers import client, embed, COLLECTION

        def retrieve(query: str, top_k: int = 5):
            query_vector = embed(query)
            results = client.search(collection_name=COLLECTION, query_vector=query_vector, limit=top_k)
            return [
                {"text": r.payload["text"], "title": r.payload["title"], "url": r.payload["url"], "score":
r.score}
                for r in results
            ]
```

Checkpoint: run `retrieve("what is retrieval augmented generation")` in a Python shell and confirm it returns genuinely relevant chunks, not random ones.

---

9. Phase 4 — Planner Agent

```
        # backend/agents/planner.py
        import anthropic
        import json
        import os

        client = anthropic.Anthropic(api_key=os.getenv("ANTHROPIC_API_KEY"))

        PLANNER_PROMPT = """You are a research planning assistant. Given a research question,
        break it into 3-5 focused sub-questions that, if answered, would together answer the
        original question. Respond ONLY with a JSON list of strings, no other text.

        Research question: {question}
        """

        def plan(question: str) -> list[str]:
            response = client.messages.create(
                model="claude-haiku-4-5-20251001",
                max_tokens=500,
                messages=[{"role": "user", "content": PLANNER_PROMPT.format(question=question)}]
            )
            text = response.content[0].text.strip()
            return json.loads(text)
```

Checkpoint: test with a real question, e.g. `plan("How does federated learning preserve privacy?")`, and confirm the sub-questions are sensible and non-overlapping.

---

10. Phase 5 — Critic Agent (Fact-Checking)

This is the most important agent for output quality — it is what prevents hallucination.

```
        # backend/agents/critic.py
        import anthropic
        import os
```

```
        client = anthropic.Anthropic(api_key=os.getenv("ANTHROPIC_API_KEY"))

        CRITIC_PROMPT = """You are a fact-checking assistant. Given a CLAIM and a set of SOURCE
        EXCERPTS, determine whether the claim is directly supported by the excerpts.

        Respond with exactly one word: SUPPORTED, PARTIALLY_SUPPORTED, or NOT_SUPPORTED,
        followed by a one-sentence justification.

        CLAIM: {claim}

        SOURCE EXCERPTS:
        {sources}
        """

        def check_claim(claim: str, sources: list[str]) -> dict:
            joined_sources = "\n---\n".join(sources)
            response = client.messages.create(
                model="claude-haiku-4-5-20251001",
                max_tokens=150,
                messages=[{"role": "user", "content": CRITIC_PROMPT.format(claim=claim,
sources=joined_sources)}]
            )
            text = response.content[0].text.strip()
            verdict = text.split()[0]
            return {"claim": claim, "verdict": verdict, "justification": text}
```

Design rule: any claim marked `NOT_SUPPORTED` must be dropped before reaching the Writer agent. This single rule is what makes MARA trustworthy rather than just another chatbot.

---

11. Phase 6 — Writer Agent

```
        # backend/agents/writer.py
        import anthropic
        import os

        client = anthropic.Anthropic(api_key=os.getenv("ANTHROPIC_API_KEY"))

        WRITER_PROMPT = """You are a research writing assistant. Using ONLY the verified findings
        below, write a clear, well-organized summary answering the original question. Cite each
        claim inline using [Source: <title>] format. Do not add any information not present in
        the verified findings.

        ORIGINAL QUESTION: {question}

        VERIFIED FINDINGS:
        {findings}
        """

        def write_summary(question: str, findings: list[dict]) -> str:
            findings_text = "\n\n".join(
                f"- {f['claim']} [Source: {f['title']}]" for f in findings
            )
            response = client.messages.create(
                model="claude-sonnet-4-6",
                max_tokens=1200,
                messages=[{"role": "user", "content": WRITER_PROMPT.format(question=question,
findings=findings_text)}]
            )
            return response.content[0].text
```

---

12. Phase 7 — LangGraph Orchestration (Wiring Everything

Together)

```
        # backend/graph.py
        from langgraph.graph import StateGraph, END
        from typing import TypedDict
        from backend.agents.planner import plan
        from backend.agents.retriever import retrieve
        from backend.agents.critic import check_claim
        from backend.agents.writer import write_summary

        class MARAState(TypedDict):
            question: str
            sub_questions: list[str]
            retrieved: list[dict]
            verified_findings: list[dict]
            final_answer: str

        def planner_node(state: MARAState) -> MARAState:
            state["sub_questions"] = plan(state["question"])
            return state

        def retriever_node(state: MARAState) -> MARAState:
            all_chunks = []
            for sub_q in state["sub_questions"]:
                chunks = retrieve(sub_q, top_k=3)
                all_chunks.extend(chunks)
            state["retrieved"] = all_chunks
            return state

        def critic_node(state: MARAState) -> MARAState:
            verified = []
            for chunk in state["retrieved"]:
                result = check_claim(chunk["text"], [chunk["text"]])
                if result["verdict"] == "SUPPORTED":
                    verified.append({"claim": chunk["text"], "title": chunk["title"], "url": chunk["url"]}
            state["verified_findings"] = verified
            return state

        def writer_node(state: MARAState) -> MARAState:
            if not state["verified_findings"]:
                state["final_answer"] = "Insufficient verified evidence was found to answer this question
reliably."
            else:
                state["final_answer"] = write_summary(state["question"], state["verified_findings"])
            return state

        def build_graph():
            graph = StateGraph(MARAState)
            graph.add_node("planner", planner_node)
            graph.add_node("retriever", retriever_node)
            graph.add_node("critic", critic_node)
            graph.add_node("writer", writer_node)

            graph.set_entry_point("planner")
            graph.add_edge("planner", "retriever")
            graph.add_edge("retriever", "critic")
            graph.add_edge("critic", "writer")
            graph.add_edge("writer", END)

            return graph.compile()

        mara_app = build_graph()

        if __name__ == "__main__":
            result = mara_app.invoke({"question": "How does federated learning preserve data privacy?"})
            print(result["final_answer"])
```

Checkpoint: running this file directly should print a complete, cited answer end-to-end. This is your first full working demo — a major milestone.

---

13. Phase 8 — Backend API (FastAPI)

```
# backend/main.py
from fastapi import FastAPI
from pydantic import BaseModel
from backend.graph import mara_app

app = FastAPI(title="MARA API")

class QueryRequest(BaseModel):
    question: str

@app.post("/query")
def query(request: QueryRequest):
    result = mara_app.invoke({"question": request.question})
    return {
        "answer": result["final_answer"],
        "sources": result["verified_findings"],
    }

@app.get("/health")
def health():
    return {"status": "ok"}
```

Run it:

```
uvicorn backend.main:app --reload --port 8000
```

Test with:

```
curl -X POST http://localhost:8000/query -H "Content-Type: application/json" \
     -d '{"question": "How does federated learning preserve data privacy?"}'
```

---

14. Phase 9 — Frontend Interface

Fastest option (Streamlit):

```
# frontend/app.py
import streamlit as st
import requests

st.title("MARA — Multi-Agent Research Assistant")
question = st.text_input("Ask a research question:")

if st.button("Search") and question:
    with st.spinner("Planning, retrieving, verifying, writing..."):
        response = requests.post("http://localhost:8000/query", json={"question": question})
        data = response.json()
    st.markdown(data["answer"])
    with st.expander("Show sources"):
        for src in data["sources"]:
            st.write(f"- **{src['title']}** — [{src['url']}]({src['url']})")
```

Run it:

```
streamlit run frontend/app.py
```

---

15. Phase 10 — Deployment (Free Tier)

Component	Free Service	Notes
Backend (FastAPI)	Render.com or Railway.app	Connect your GitHub repo; free tier sleeps after inactivity — acceptable for a demo
Frontend (Streamlit)	Streamlit Community Cloud	One-click deploy directly from GitHub
Vector DB	Qdrant Cloud	Free cluster tier is sufficient for a few thousand chunks
Environment variables	Set via each platform's dashboard	Never commit `.env` to GitHub

Basic deployment steps: 1. Push your repo to GitHub (public or private). 2. On Render: New → Web Service → connect repo → set start command `uvicorn backend.main:app --host 0.0.0.0 --port $PORT`. 3. On Streamlit Cloud: New app → connect repo → point to `frontend/app.py` → set the backend URL as a secret/environment variable (update `frontend/app.py` to read it via `os.getenv` instead of hardcoding `localhost`).

---

16. Phase 11 — Evaluation Framework

This is what turns "I built an app" into "I built and evaluated a system" — essential for your report and viva.

16.1 Build a Benchmark Set

Create `evaluation/benchmark_queries.json` with 15–20 real research questions across different topics (e.g., federated learning, RAG, agentic AI, edge computing).

16.2 Metrics to Report

Metric	What It Measures	How to Compute
Retrieval Precision	Are retrieved chunks actually relevant?	Manually label top-k chunks as relevant/irrelevant for each benchmark query
Factual Consistency Rate	% of claims marked SUPPORTED by the Critic	Log Critic verdicts across all benchmark runs
Citation Accuracy	Do cited sources actually contain the claim?	Manual spot-check of a sample
Latency	Time from query to final answer	Wrap `mara_app.invoke()` with a timer
Baseline Comparison	MARA vs. a single naive LLM prompt on the same questions	Run both, compare factual consistency and depth

```
# evaluation/evaluate.py
import time
import json
from backend.graph import mara_app

with open("evaluation/benchmark_queries.json") as f:
    queries = json.load(f)

results = []
for q in queries:
    start = time.time()
    output = mara_app.invoke({"question": q})
    latency = time.time() - start
    results.append({
```

```
                "question": q,
                "num_sources": len(output["verified_findings"]),
                "latency_sec": round(latency, 2),
                "answer": output["final_answer"],
            })

        with open("evaluation/results.json", "w") as f:
            json.dump(results, f, indent=2)

        print(f"Evaluated {len(results)} queries. Average latency: "
              f"{sum(r['latency_sec'] for r in results) / len(results):.2f}s")
```

Important for your report: always include the naive single-prompt baseline comparison — this is what proves the multi-agent design adds real value, not just complexity.

---

17. Phase 12 — Testing, Report & Viva Preparation

- Test with at least 15–20 diverse questions before your final demo. Note failure cases honestly in your report — evaluators respect honest limitation sections.
- Prepare a **live, un-scripted demo** — type a question your panel suggests on the spot. This is the single most convincing thing you can do.
- Prepare answers to likely questions:
  - “Why not just use one LLM call?” → explain hallucination risk and lack of traceability.
  - “How do you know the Critic agent works?” → show your factual-consistency-rate metric.
  - “What happens if retrieval finds nothing relevant?” → show your fallback message (“insufficient evidence”).
  - “Why LangGraph over other frameworks?” → explicit state graph is easier to debug and reason about than fully autonomous agent loops.

---

18. Full Repository Structure (Reference)

```
mara-project/
├── .env
├── .gitignore
├── requirements.txt
├── README.md
├── backend/
│   ├── main.py
│   ├── config.py
│   ├── graph.py
│   ├── agents/
│   │   ├── planner.py
│   │   ├── retriever.py
│   │   ├── critic.py
│   │   └── writer.py
│   └── ingestion/
│       ├── fetch_papers.py
│       └── index_papers.py
├── frontend/
│   └── app.py
└── evaluation/
    ├── benchmark_queries.json
    ├── results.json
    └── evaluate.py
```

---

19. Troubleshooting Guide

Problem	Likely Cause	Fix

Retriever returns irrelevant chunks	Chunk size too large/small, or too few papers indexed	Reduce chunk size to ~300-500 words; index more papers per topic
Critic marks everything NOT_SUPPORTED	Prompt too strict, or comparing claim against unrelated chunk	Loosen prompt wording slightly; verify retrieval quality first
LangGraph state errors	Missing key in `TypedDict` or node not returning full state	Ensure every node returns the complete state dict, not just the modified field
API rate limit errors during testing	Too many rapid calls during development	Cache repeated test queries locally; use a cheaper/smaller model while iterating
Streamlit can't reach backend after deployment	Hardcoded `localhost` URL	Replace with an environment variable pointing to your deployed backend URL
Qdrant collection not found	Forgot to run indexing script before querying	Run `index_papers.py` once before starting the API

---

20. Cost Breakdown

Item	Cost
Anthropic/OpenAI API calls (development)	Free trial credits typically sufficient
Qdrant Cloud free tier	$0
Render/Railway free tier	$0
Streamlit Community Cloud	$0
Total realistic cost	**₹0–1,500** (only if trial credits run out near your deadline)

---

21. Week-by-Week Timeline

Weeks	Focus
1–2	Environment setup, data ingestion pipeline (Phase 1–2)
3–4	Retriever agent + indexing (Phase 3)
5–6	Planner agent (Phase 4)
6–7	Critic agent (Phase 5)
7–9	Writer agent + full LangGraph orchestration (Phase 6–7)
10–12	Backend API + frontend (Phase 8–9)
12–13	Deployment (Phase 10)
13–15	Evaluation framework + baseline comparison (Phase 11)
15–16	Testing, report writing, viva preparation (Phase 12)

---

22. Base Paper & Further Reading

Primary base paper: Flores, N., Ramirez, C., Mauricio, D. "Expert Multi-Agent Conversational System Using Retrieval-Augmented Generation and Dynamic Text-to-SQL for Government Transparency." *IEEE Access*, vol. 13, 2025.

Learning resources (all free): - LangGraph official documentation and tutorials - Anthropic's prompt engineering guide (docs.claude.com) - Qdrant documentation and quickstart - DeepLearning.AI short course: "AI Agents in LangGraph" - DeepLearning.AI short course: "LangChain for LLM Application

Development”

This guide is a living document — as you build, you will discover project-specific edge cases beyond what’s covered here. Keep notes as you go; they become the “Challenges Faced” section of your final report.